

Module 9) Python DB and Framework

Practical

1. HTML in Python

Q.1 Write a Python program to render an HTML file using Django's template system.

Ans.

Step 1: Create Django Project

```
django-admin startproject myproject
```

```
cd myproject
```

```
python manage.py startapp myapp
```

Step 2: Add App to settings.py

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'myapp',  
]
```

Step 3: Create Template Folder

Create folders:

```
myapp/
```

```
└── templates/
```

```
    └── index.html
```

Step 4: HTML File (index.html)

```
<!DOCTYPE html>

<html>

<head>

    <title>Django Template</title>

</head>

<body>

    <h1>{{ message }}</h1>

</body>

</html>
```

Step 5: Create View (views.py)

```
from django.shortcuts import render

def home(request):

    return render(request, 'index.html', {'message': 'Welcome to Django Template System'})
```

Step 6: Configure URL (urls.py)

Project urls.py

```
from django.contrib import admin

from django.urls import path

from myapp import views

urlpatterns = [

    path('admin/', admin.site.urls),

    path("", views.home, name='home'),

]
```

Step 7: Run Server

```
python manage.py runserver
```

Open browser:

<http://127.0.0.1:8000/>

Practical Example:

1. Write a Django project that renders an HTML file displaying “Welcome to Doctor Finder” on the home page.

Ans. Step 1: Create Django Project and App

```
django-admin startproject doctorfinder
```

```
cd doctorfinder
```

```
python manage.py startapp home
```

Step 2: Add App in settings.py

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'home',  
]
```

Step 3: Create Templates Folder

```
home/
```

```
└── templates/
```

```
    └── index.html
```

Step 4: HTML File (index.html)

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <title>Doctor Finder</title>
```

```
</head>
```

```
<body>
```

```
<h1>Welcome to Doctor Finder</h1>
</body>
</html>
```

Step 5: Create View (home/views.py)

```
from django.shortcuts import render
```

```
def home_page(request):
    return render(request, 'index.html')
```

Step 6: Configure URLs

Project urls.py (doctorfinder/urls.py)

```
from django.contrib import admin
from django.urls import path
from home import views
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path("", views.home_page, name='home'),
]
```

Step 7: Run the Server

```
python manage.py runserver
```

Open browser:

<http://127.0.0.1:8000/>

2. CSS in Python

Q.2 Create a CSS file to style a basic HTML template in Django.

Ans.

Step 1: Configure Static Files in settings.py

```
STATIC_URL = '/static/'
```

```
STATICFILES_DIRS = [  
    BASE_DIR / "static",  
]
```

Step 2: Create Static Folder Structure

```
project/  
├── static/  
│   ├── css/  
│   │   └── style.css  
└── app/  
    ├── templates/  
    │   └── index.html
```

Step 3: Create CSS File (static/css/style.css)

```
body {  
    background-color: #f2f2f2;  
    font-family: Arial, sans-serif;  
}
```

```
h1 {  
    color: #2c3e50;  
    text-align: center;  
    margin-top: 100px;
```

```
}
```

```
p {  
    text-align: center;  
    font-size: 18px;  
    color: #555;  
}
```

Step 4: Link CSS in HTML Template (index.html)

```
{% load static %}  
<!DOCTYPE html>  
<html>  
<head>  
    <title>Django CSS Example</title>  
    <link rel="stylesheet" href="{% static 'css/style.css' %}">  
</head>  
<body>  
  
    <h1>Welcome to Doctor Finder</h1>  
    <p>Find the best doctors near you</p>  
  
</body>  
</html>
```

Practical Example:

2) Write a Django project to display a webpage with custom CSS styling for a doctor profile page.

Ans.

Step 1: Create Django Project and App

```
django-admin startproject doctorprofile
```

```
cd doctorprofile
```

```
python manage.py startapp profile
```

Step 2: Add App in settings.py

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'profile',  
]
```

Step 3: Configure Static Files (settings.py)

```
STATIC_URL = '/static/'
```

```
STATICFILES_DIRS = [  
    BASE_DIR / "static",  
]
```

Step 4: Create Folder Structure

doctorprofile/

```
|— static/
|   |— css/
|       |— doctor.css
|— profile/
|   |— templates/
|       |— doctor.html
|       |— views.py
|— doctorprofile/
    |— urls.py
```

Step 5: Create CSS File (static/css/doctor.css)

```
body {
    background-color: #f4f6f7;
    font-family: Arial, sans-serif;
}

.profile-card {
    width: 400px;
    margin: 80px auto;
    padding: 20px;
    background: white;
    border-radius: 10px;
    box-shadow: 0px 4px 10px rgba(0,0,0,0.1);
    text-align: center;
}
```

```
.profile-card h1 {  
    color: #2c3e50;  
}
```

```
.profile-card p {  
    font-size: 16px;  
    color: #555;  
}
```

```
.specialization {  
    color: #1abc9c;  
    font-weight: bold;  
}
```

Step 6: Create HTML Template (doctor.html)

```
{% load static %}  
<!DOCTYPE html>  
<html>  
<head>  
    <title>Doctor Profile</title>  
    <link rel="stylesheet" href="{% static 'css/doctor.css' %}">  
</head>  
<body>  
  
<div class="profile-card">  
    <h1>Dr. Rahul Sharma</h1>  
    <p class="specialization">Cardiologist</p>  
    <p>Experience: 10 Years</p>
```

```
<p>Location: New Delhi</p>
</div>
```

```
</body>
</html>
```

Step 7: Create View (profile/views.py)

```
from django.shortcuts import render
```

```
def doctor_profile(request):
    return render(request, 'doctor.html')
```

Step 8: Configure URLs (doctorprofile/urls.py)

```
from django.contrib import admin
from django.urls import path
from profile import views
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path("", views.doctor_profile, name='doctor_profile'),
]
```

Step 9: Run the Server

```
python manage.py runserver
```

3. JavaScript with Python

Q.3 Create a Django project with JavaScript-enabled form validation.

Ans.

Step 1: Create Django Project and App

```
django-admin startproject jsvalidation
```

```
cd jsvalidation
```

```
python manage.py startapp formapp
```

Step 2: Add App in settings.py

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'formapp',  
]
```

Step 3: Configure Static Files (settings.py)

```
STATIC_URL = '/static/'
```

```
STATICFILES_DIRS = [  
    BASE_DIR / "static",  
]
```

Step 4: Folder Structure

jsvalidation/

```
|— static/
|   |— js/
|       — validate.js
|— formapp/
|   |— templates/
|       — form.html
|   — views.py
— jsvalidation/
    — urls.py
```

Step 5: Create HTML Form (form.html)

```
{% load static %}

<!DOCTYPE html>

<html>

<head>

    <title>Form Validation</title>

    <script src="{% static 'js/validate.js' %}" "></script>

</head>

<body>

<h2>User Registration</h2>

<form onsubmit="return validateForm()">

    <label>Name:</label><br>

    <input type="text" id="name"><br><br>
```

```
<label>Email:</label><br>
<input type="text" id="email"><br><br>

<label>Password:</label><br>
<input type="password" id="password"><br><br>

<input type="submit" value="Submit">
</form>

<p id="error" style="color:red;"></p>

</body>
</html>
```

Step 6: Create JavaScript File (static/js/validate.js)

```
function validateForm() {
    var name = document.getElementById("name").value;
    var email = document.getElementById("email").value;
    var password = document.getElementById("password").value;
    var error = document.getElementById("error");

    if (name === "" || email === "" || password === "") {
        error.innerHTML = "All fields are required!";
        return false;
    }

    if (password.length < 6) {
        error.innerHTML = "Password must be at least 6 characters long!";
    }
}
```

```
        return false;
    }

    alert("Form submitted successfully!");
    return true;
}
```

Step 7: Create View (formapp/views.py)

```
from django.shortcuts import render
```

```
def form_page(request):
    return render(request, 'form.html')
```

Step 8: Configure URLs (jsvalidation/urls.py)

```
from django.contrib import admin
from django.urls import path
from formapp import views
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path("", views.form_page, name='form'),
]
```

Step 9: Run the Server

```
python manage.py runserver
```

Open browser:

<http://127.0.0.1:8000/>

Practical Example:

3) Write a Django project where JavaScript is used to validate a patient registration form on the client side.

Ans.

Step 1: Create Django Project and App

```
django-admin startproject patient_portal
```

```
cd patient_portal
```

```
python manage.py startapp patients
```

Step 2: Add App to Settings

settings.py

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'patients',  
]
```

Step 3: Create Patient Model

patients/models.py

```
from django.db import models
```

```
class Patient(models.Model):  
    name = models.CharField(max_length=100)  
    email = models.EmailField(unique=True)
```

```
phone = models.CharField(max_length=10)
age = models.IntegerField()
```

```
def __str__(self):
    return self.name
```

Step 4: Apply Migrations

```
python manage.py makemigrations
python manage.py migrate
```

Step 5: Create Patient Registration View

patients/views.py

```
from django.shortcuts import render, redirect
from .models import Patient

def register(request):
    if request.method == "POST":
        Patient.objects.create(
            name=request.POST['name'],
            email=request.POST['email'],
            phone=request.POST['phone'],
            age=request.POST['age']
        )
        return redirect('success')
    return render(request, 'patients/register.html')

def success(request):
    return render(request, 'patients/success.html')
```

Step 6: Configure URLs

patients/urls.py

```
from django.urls import path
from .views import register, success

urlpatterns = [
    path("", register, name='register'),
    path('success/', success, name='success'),
]
```

Project urls.py

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path("", include('patients.urls')),
]
```

Step 7: Create Registration Template with JavaScript Validation

templates/patients/register.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Patient Registration</title>
    <script>
        function validateForm() {
```

```
let name = document.forms["patientForm"]["name"].value;
let email = document.forms["patientForm"]["email"].value;
let phone = document.forms["patientForm"]["phone"].value;
let age = document.forms["patientForm"]["age"].value;

if (name === "") {
    alert("Name must be filled out");
    return false;
}

if (!email.includes("@")) {
    alert("Enter a valid email address");
    return false;
}

if (phone.length !== 10 || isNaN(phone)) {
    alert("Enter a valid 10-digit phone number");
    return false;
}

if (age <= 0) {
    alert("Enter a valid age");
    return false;
}

return true;
}
```

```
</script>
</head>
<body>

<h2>Patient Registration Form</h2>

<form name="patientForm" method="post" onsubmit="return validateForm()">
  {% csrf_token %}
  Name: <input type="text" name="name"><br><br>
  Email: <input type="text" name="email"><br><br>
  Phone: <input type="text" name="phone"><br><br>
  Age: <input type="number" name="age"><br><br>
  <button type="submit">Register</button>
</form>

</body>
</html>
```

Step 8: Create Success Page

templates/patients/success.html

```
<h2>Registration Successful!</h2>
<p>Patient data has been saved.</p>
```

Step 9: Run the Server

```
python manage.py runserver
```

4. Django Introduction

Q.4 Write a short project using Django's built-in tools to render a simple webpage.

Ans.

Step 1: Create Django Project and App

```
django-admin startproject simpleproject
```

```
cd simpleproject
```

```
python manage.py startapp main
```

Step 2: Add App in settings.py

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'main',  
]
```

Step 3: Create HTML Template

Create folder and file:

```
main/templates/index.html
```

index.html

```
<!DOCTYPE html>  
  
<html>  
  
<head>  
    <title>Simple Page</title>  
</head>  
  
<body>
```

```
<h1>Welcome to Django</h1>
<p>This is a simple webpage rendered using Django.</p>
</body>
</html>
```

Step 4: Create View (views.py)

```
from django.shortcuts import render
```

```
def home(request):
    return render(request, 'index.html')
```

Step 5: Configure URLs (urls.py)

Project urls.py

```
from django.contrib import admin
from django.urls import path
from main import views
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path("", views.home, name='home'),
]
```

Step 6: Run the Server

```
python manage.py runserver
```

Output

Welcome to Django

This is a simple webpage rendered using Django.

Practical Example:

4) Write a Python program to create a Django project and understand its directory structure.

Step 1: Create Django Project

`django-admin startproject myproject`

Step 2: Project Directory Structure

`myproject/`

```
|— manage.py
|— myproject/
    |— __init__.py
    |— asgi.py
    |— settings.py
    |— urls.py
    |— wsgi.py
```

Step 3: Explanation of Files and Folders

1. manage.py

- A Python file used to run Django administrative tasks.
- Commands like `runserver`, `migrate`, `createsuperuser` are executed using this file.

2. Inner myproject Folder

This folder contains the core configuration of the Django project.

a) __init__.py

- Marks the directory as a Python package.

b) settings.py

- Main configuration file.
- Contains settings like database, installed apps, templates, static files.

c) urls.py

- Controls URL routing.

- Maps URLs to views.

d) wsgi.py

- Used for deploying Django applications on production servers.

e) asgi.py

- Used for asynchronous web servers.
- Supports WebSockets and async features.

Step 4: Run the Django Server

```
python manage.py runserver
```

5. Virtual Environment

Q.5 Set up a virtual environment for a Django project.

Step 1: Check Python Installation

```
python --version
```

Step 2: Create a Virtual Environment

```
python -m venv myenv
```

- venv is Python's built-in tool to create virtual environments.
- myenv is the virtual environment name.

Step 3: Activate the Virtual Environment

On Windows:

```
myenv\Scripts\activate
```

On Linux / macOS:

```
source myenv/bin/activate
```

Step 4: Install Django Inside the Virtual Environment

```
pip install Django
```

Check Django version:

```
django-admin --version
```

Step 5: Create Django Project

```
django-admin startproject myproject
```

```
cd myproject
```

Step 6: Deactivate Virtual Environment

Deactivate

Directory Structure After Setup

myenv/

myproject/

```
|— manage.py
|— myproject/
    |— settings.py
    |— urls.py
    |— wsgi.py
    |— asgi.py
```

Practical Example:

5) Write a Python program to create and activate a virtual environment, then install Django in it.

Ans.

Step 1: Check Python Installation

```
python --version
```

Step 2: Create a Virtual Environment

```
python -m venv myenv
```

- venv → Python's built-in virtual environment module
- myenv → Name of the virtual environment

Step 3: Activate the Virtual Environment

On Windows:

```
myenv\Scripts\activate
```

On Linux / macOS:

`source myenv/bin/activate`

Step 4: Install Django Inside the Virtual Environment

`pip install Django`

Step 5: Verify Django Installation

`django-admin --version`

Step 6: (Opt) Create a Django Project

`django-admin startproject myproject`

Step 7: Deactivate the Virtual Environment

Deactivate

Directory Structure

`myenv/`

`myproject/`

├── `manage.py`

└── `myproject/`

 ├── `settings.py`

 ├── `urls.py`

 ├── `wsgi.py`

 └── `asgi.py`

6. Project and App Creation

Q.6 Create a Django project with an app to manage doctor profiles.

Ans.

Step 1: Create Project and App

```
django-admin startproject doctorfinder
```

```
cd doctorfinder
```

```
python manage.py startapp doctors
```

Step 2: Add App in settings.py

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'doctors',  
]
```

Step 3: Create Doctor Model (doctors/models.py)

```
from django.db import models
```

```
class Doctor(models.Model):  
    name = models.CharField(max_length=100)  
    specialization = models.CharField(max_length=100)  
    experience = models.IntegerField()  
    location = models.CharField(max_length=100)  
    def __str__(self):  
        return self.name
```

Step 4: Apply Migrations

```
python manage.py makemigrations
```

```
python manage.py migrate
```

Step 5: Register Model in Admin (doctors/admin.py)

```
from django.contrib import admin
```

```
from .models import Doctor
```

```
admin.site.register(Doctor)
```

Create admin user:

```
python manage.py createsuperuser
```

Step 6: Create View (doctors/views.py)

```
from django.shortcuts import render
```

```
from .models import Doctor
```

```
def doctor_list(request):
```

```
    doctors = Doctor.objects.all()
```

```
    return render(request, 'doctor_list.html', {'doctors': doctors})
```

Step 7: Create Template

Create folder:

```
doctors/templates/doctor_list.html
```

doctor_list.html

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <title>Doctor Profiles</title>
```

```
</head>
```

```
<body>
```

```

<h1>Doctor Profiles</h1>

{% for doctor in doctors %}
    <div>
        <h3>{{ doctor.name }}</h3>
        <p>Specialization: {{ doctor.specialization }}</p>
        <p>Experience: {{ doctor.experience }} years</p>
        <p>Location: {{ doctor.location }}</p>
        <hr>
    </div>
{% empty %}
    <p>No doctors available.</p>
{% endfor %}
</body>
</html>

```

Step 8: Configure URL

App urls.py (create file)

```
from django.urls import path
```

```
from . import views
```

```
urlpatterns = [
    path("", views.doctor_list, name='doctor_list'),
]
```

Project urls.py

```
from django.contrib import admin  
from django.urls import path, include
```

```
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path("", include('doctors.urls')),  
]
```

Step 9: Run Server

```
python manage.py runserver
```

Practical Example:

6) Write a Python program to create a Django project and a new app within the project called doctor.

Ans.

Step 1: Create Django Project

```
django-admin startproject hospital  
cd hospital
```

Step 2: Create a New App Name doctor

```
python manage.py startapp doctor
```

Step 3: Project and App Directory Structure

hospital/

```
|— manage.py
|— hospital/
|   |— __init__.py
|   |— settings.py
|   |— urls.py
|   |— asgi.py
|   |— wsgi.py
|— doctor/
|   |— __init__.py
|   |— admin.py
|   |— apps.py
|   |— models.py
|   |— tests.py
|   |— views.py
|   |— migrations/
|       |— __init__.py
```

Step 4: Register the App in settings.py

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
```

```
'doctor',  
]
```

Step 5: Run the Django Server

```
python manage.py runserver
```

7. MVT Pattern Architecture

Q.7 Build a simple Django app showcasing how the MVT architecture works.

Ans.

What is MVT

- Model → Handles database (data structure)
- View → Contains business logic
- Template → Handles presentation (HTML)

Step 1: Create Django Project and App

```
django-admin startproject mvtpproject
```

```
cd mvtpproject
```

```
python manage.py startapp demo
```

Step 2: Register App in settings.py

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'demo',  
]
```

Step 3: Model

models.py

```
from django.db import models
```

```
class Message(models.Model):  
    text = models.CharField(max_length=100)
```

```
    def __str__(self):  
        return self.text
```

Apply migrations:

```
python manage.py makemigrations
```

```
python manage.py migrate
```

Step 4: View

views.py

```
from django.shortcuts import render  
from .models import Message
```

```
def home(request):  
    msg = Message.objects.first()  
    return render(request, 'home.html', {'message': msg})
```

Step 5: Template

Create folder:

```
templates/home.html
```

home.html

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>MVT Example</title>
</head>
<body>
    <h1>Django MVT Architecture</h1>
    <p>{{ message.text }}</p>
</body>
</html>
```

Step 6: URL

urls.py

```
from django.contrib import admin
from django.urls import path
from demo import views
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path("", views.home, name='home'),
]
```

Step 7: Add Data Using Admin

```
python manage.py createsuperuser
```

- Login to /admin
- Add a message like: “Welcome to Django MVT Architecture”

Step 8: Run Server

```
python manage.py runserver
```

Practical Example:

7) Write a Django project with models, views, and templates to display doctor information.

Ans.

Step 1: Create Django Project and App

```
django-admin startproject doctor_finder
```

```
cd doctor_finder
```

```
python manage.py startapp doctors
```

Step 2: Add App to Settings

settings.py

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'doctors',  
]
```

Step 3: Create Doctor Model

doctors/models.py

```
from django.db import models
```

```
class Doctor(models.Model):
```

```
    name = models.CharField(max_length=100)
```

```
    specialty = models.CharField(max_length=100)
```

```
experience = models.IntegerField()
contact = models.CharField(max_length=15)
```

```
def __str__(self):
    return self.name
```

Step 4: Apply Migrations

```
python manage.py makemigrations
python manage.py migrate
```

Step 5: Register Model in Admin

doctors/admin.py

```
from django.contrib import admin
from .models import Doctor
```

```
admin.site.register(Doctor)
```

Create admin user:

```
python manage.py createsuperuser
```

Step 6: Insert Doctor Data

<http://127.0.0.1:8000/admin>

Add doctors like:

- Name: Dr. Rahul Sharma
- Specialty: Cardiologist
- Experience: 10
- Contact: 9876543210

Step 7: Create View to Fetch Doctor Data

doctors/views.py

```
from django.shortcuts import render
from .models import Doctor

def doctor_list(request):
    doctors = Doctor.objects.all()
    return render(request, 'doctors/doctor_list.html', {'doctors': doctors})
```

Step 8: Configure URLs

doctors/urls.py

```
from django.urls import path
from .views import doctor_list

urlpatterns = [
    path("", doctor_list, name='doctor_list'),
]
```

Project urls.py

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path("", include('doctors.urls')),
]
```

Step 9: Create Template to Display Doctor Information

templates/doctors/doctor_list.html

```
<!DOCTYPE html>

<html>

<head>

    <title>Doctor Information</title>

</head>

<body>


<h2>Doctor Finder</h2>


<table border="1">

    <tr>

        <th>Name</th>

        <th>Specialty</th>

        <th>Experience (Years)</th>

        <th>Contact</th>

    </tr>


    {% for doctor in doctors %}

    <tr>

        <td>{{ doctor.name }}</td>

        <td>{{ doctor.specialty }}</td>

        <td>{{ doctor.experience }}</td>

        <td>{{ doctor.contact }}</td>

    </tr>

    {% empty %}
```

```
<tr>
    <td colspan="4">No doctors available</td>
</tr>

{% endfor %}
</table>

</body>
</html>
```

Step 10: Run the Server

```
python manage.py runserver
```

8. Django Admin Panel

Q.8 Set up and customize the Django admin panel to manage a "Doctor Finder" project.

Ans. Step 1: Create Project and App

```
django-admin startproject doctorfinder
```

```
cd doctorfinder
```

```
python manage.py startapp doctors
```

Step 2: Register App in settings.py

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
```

```
    'doctors',  
]
```

Step 3: Create Doctor Model

doctors/models.py

```
from django.db import models  
  
class Doctor(models.Model):  
    name = models.CharField(max_length=100)  
    specialization = models.CharField(max_length=100)  
    experience = models.IntegerField()  
    hospital = models.CharField(max_length=100)  
    available = models.BooleanField(default=True)  
    def __str__(self):  
        return self.name
```

Step 4: Apply Migrations

```
python manage.py makemigrations  
python manage.py migrate
```

Step 5: Create Superuser

```
python manage.py createsuperuser
```

Step 6: Register Model in Admin Panel

doctors/admin.py

```
from django.contrib import admin  
from .models import Doctor
```

```
@admin.register(Doctor)
```

```
class DoctorAdmin(admin.ModelAdmin):  
    list_display = ('name', 'specialization', 'experience', 'hospital', 'available')  
    list_filter = ('specialization', 'available')
```

```
search_fields = ('name', 'hospital')
```

```
list_editable = ('available',)
```

Step 7: Customize Admin Panel Title

doctorfinder/admin.py

```
from django.contrib import admin
```

```
admin.site.site_header = "Doctor Finder Admin"
```

```
admin.site.site_title = "Doctor Finder Portal"
```

```
admin.site.index_title = "Welcome to Doctor Finder Administration"
```

Step 8: Run Server

```
python manage.py runserver
```

Practical Example:

8) Write a Django project to create an admin panel and add custom fields for managing doctor information.

Ans.

Step 1: Create Django Project and App

```
django-admin startproject doctor_finder
```

```
cd doctor_finder
```

```
python manage.py startapp doctors
```

Step 2: Add App to Settings

settings.py

```
INSTALLED_APPS = [
```

```
    'django.contrib.admin',
```

```
    'django.contrib.auth',
```

```
    'django.contrib.contenttypes',
```

```
'django.contrib.sessions',  
'django.contrib.messages',  
'django.contrib.staticfiles',  
'doctors',  
]
```

Step 3: Create Doctor Model with Custom Fields

doctors/models.py

```
from django.db import models
```

```
class Doctor(models.Model):  
    name = models.CharField(max_length=100)  
    specialty = models.CharField(max_length=100)  
    experience = models.IntegerField(help_text="Years of experience")  
    available_days = models.CharField(max_length=100)  
    contact_number = models.CharField(max_length=15)  
    email = models.EmailField(unique=True)  
  
    def __str__(self):  
        return self.name
```

Step 4: Apply Migrations

```
python manage.py makemigrations
```

```
python manage.py migrate
```

Step 5: Customize Django Admin Panel

doctors/admin.py

```
from django.contrib import admin
```

```
from .models import Doctor
```

```
@admin.register(Doctor)
```

```
class DoctorAdmin(admin.ModelAdmin):
```

```
    list_display = ('name', 'specialty', 'experience', 'contact_number')
```

```
    list_filter = ('specialty',)
```

```
    search_fields = ('name', 'specialty', 'email')
```

```
    ordering = ('name',)
```

Step 6: Create Superuser

```
python manage.py createsuperuser
```

Step 7: Run the Server

```
python manage.py runserver
```

Open admin panel:

```
http://127.0.0.1:8000/admin
```

9. URL Patterns and Template Integration

Q.9 Create a Django project with URL patterns and corresponding views and templates.

Ans.

Step 1: Create Django Project and App

```
django-admin startproject urlproject
```

```
cd urlproject
```

```
python manage.py startapp pages
```

Step 2: Register App in settings.py

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'pages',  
]
```

Step 3: Create Templates

Create folders:

```
pages/
```

```
└── templates/
```

```
    ├── home.html
```

```
    └── about.html
```

home.html

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Home Page</title>
</head>
<body>
    <h1>Welcome to Django Home Page</h1>
    <p>This page is rendered using URL, View, and Template.</p>
</body>
</html>
```

about.html

```
<!DOCTYPE html>
<html>
<head>
    <title>About Page</title>
</head>
<body>
    <h1>About Us</h1>
    <p>This is the About page of our Django project.</p>
</body>
</html>
```

Step 4: Create Views

pages/views.py

```
from django.shortcuts import render

def home(request):
    return render(request, 'home.html')

def about(request):
    return render(request, 'about.html')
```

Step 5: Configure URL

App URL (pages/url.py) – create this file

```
from django.urls import path
```

```
from . import views
```

```
urlpatterns = [  
    path("", views.home, name='home'),  
    path('about/', views.about, name='about'),  
]
```

Project URL (urlproject/url.py)

```
from django.contrib import admin
```

```
from django.urls import path, include
```

```
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path("", include('pages.urls')),  
]
```

Step 6: Run the Server

```
python manage.py runserver
```

Practical Example:

9) Write a Django project where URL routing is used to navigate between different pages of a “Doctor Finder” site (home, profile, contact).

Ans.

Step 1: Create Django Project and App

```
django-admin startproject doctorfinder
```

```
cd doctorfinder
```

```
python manage.py startapp sitepages
```

Step 2: Register App in settings.py

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'sitepages',  
]
```

Step 3: Create Templates

Create folder:

sitepages/

└─ templates/

 └─ home.html

 └─ profile.html

 └─ contact.html

home.html

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <title>Doctor Finder - Home</title>
```

```
</head>
```

```
<body>
```

```
    <h1>Welcome to Doctor Finder</h1>
```

```
    <a href="/profile/">Doctor Profile</a> |
```

```
    <a href="/contact/">Contact Us</a>
```

```
</body>
```

```
</html>
```

profile.html

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Doctor Profile</title>
```

```
</head>
```

```
<body>
```

```
  <h1>Doctor Profile</h1>
```

```
  <p>Name: Dr. Amit Verma</p>
```

```
  <p>Specialization: Orthopedic</p>
```

```
  <p>Experience: 8 Years</p>
```

```
  <a href="/">Back to Home</a>
```

```
</body>
```

```
</html>
```

contact.html

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Contact</title>
```

```
</head>
```

```
<body>
```

```
  <h1>Contact Doctor Finder</h1>
```

```
  <p>Email: support@doctorfinder.com</p>
```

```
<p>Phone: +91 9876543210</p>
<a href="/">Back to Home</a>
</body>
</html>
```

Step 4: Create Views

sitepages/views.py

```
from django.shortcuts import render
```

```
def home(request):
    return render(request, 'home.html')
```

```
def profile(request):
    return render(request, 'profile.html')
```

```
def contact(request):
    return render(request, 'contact.html')
```

Step 5: Configure URL

App URL (sitepages/url.py)

```
from django.urls import path
```

```
from . import views
```

```
urlpatterns = [
    path("", views.home, name='home'),
    path('profile/', views.profile, name='profile'),
    path('contact/', views.contact, name='contact'),
]
```

Project URLs (doctorfinder/urls.py)

```
from django.contrib import admin
from django.urls import path, include
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path("", include('sitepages.urls')),
]
```

Step 6: Run the Server

```
python manage.py runserver
```

10. Form Validation using JavaScript

Q.10 Write a Django project to implement JavaScript form validation for a user registration form.

Ans.

Step 1: Create Django Project and App

```
django-admin startproject registration
```

```
cd registration
```

```
python manage.py startapp users
```

Step 2: Register App in settings.py

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'users',
]
```

Step 3: Configure Static Files (settings.py)

```
STATIC_URL = '/static/'
```

```
STATICFILES_DIRS = [  
    BASE_DIR / "static",  
]
```

Step 4: Project Folder Structure

registration/

```
|— static/  
|   |— js/  
|       |— validate.js  
|— users/  
|   |— templates/  
|       |— register.html  
|   |— views.py  
|— registration/  
    |— urls.py
```

Step 5: Create HTML Registration Form

users/templates/register.html

```
{% load static %}  
  
<!DOCTYPE html>  
  
<html>  
  
<head>  
    <title>User Registration</title>  
    <script src="{% static 'js/validate.js' %}"></script>  
</head>
```

```
<body>
```

```
<h2>User Registration</h2>
```

```
<form onsubmit="return validateForm()">
```

```
  <label>Username:</label><br>
```

```
  <input type="text" id="username"><br><br>
```

```
  <label>Email:</label><br>
```

```
  <input type="text" id="email"><br><br>
```

```
  <label>Password:</label><br>
```

```
  <input type="password" id="password"><br><br>
```

```
  <label>Confirm Password:</label><br>
```

```
  <input type="password" id="confirm"><br><br>
```

```
  <input type="submit" value="Register">
```

```
</form>
```

```
<p id="error" style="color:red;"></p>
```

```
</body>
```

```
</html>
```

Step 6: Create JavaScript Validation File

static/js/validate.js

```
function validateForm() {  
    var username = document.getElementById("username").value;  
    var email = document.getElementById("email").value;  
    var password = document.getElementById("password").value;  
    var confirm = document.getElementById("confirm").value;  
    var error = document.getElementById("error");  
  
    if (username === "" || email === "" || password === "" || confirm === "") {  
        error.innerHTML = "All fields are required!";  
        return false;  
    }  
  
    if (password.length < 6) {  
        error.innerHTML = "Password must be at least 6 characters!";  
        return false;  
    }  
  
    if (password !== confirm) {  
        error.innerHTML = "Passwords do not match!";  
        return false;  
    }  
  
    alert("Registration successful!");  
    return true;  
}
```

Step 7: Create View

users/views.py

```
from django.shortcuts import render
```

```
def register(request):  
    return render(request, 'register.html')
```

Step 8: Configure URLs

App URLs (users/urls.py)

```
from django.urls import path  
from . import views
```

```
urlpatterns = [  
    path("", views.register, name='register'),  
]
```

Project URLs (registration/urls.py)

```
from django.contrib import admin  
from django.urls import path, include
```

```
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path("", include('users.urls')),  
]
```

Step 9: Run the Server

```
python manage.py runserver
```

Practical Example:

10) Write a Django project that uses JavaScript to validate fields like email and phone number in a registration form.

Ans.

Step 1: Create Django Project and App

```
django-admin startproject regvalidation
```

```
cd regvalidation
```

```
python manage.py startapp accounts
```

Step 2: Register App in settings.py

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'accounts',  
]
```

Step 3: Configure Static Files (settings.py)

```
STATIC_URL = '/static/'
```

```
STATICFILES_DIRS = [  
    BASE_DIR / "static",  
]
```

Step 4: Folder Structure

regvalidation/

```
|— static/
|   |— js/
|       |— validate.js
|— accounts/
|   |— templates/
|       |— register.html
|       |— views.py
|— regvalidation/
|   |— urls.py
```

Step 5: Create Registration Form (HTML Template)

accounts/templates/register.html

```
{% load static %}

<!DOCTYPE html>

<html>

<head>

    <title>Registration Form</title>

    <script src="{% static 'js/validate.js' %}" "></script>

</head>

<body>

    <h2>User Registration</h2>

    <form onsubmit="return validateForm()">

        <label>Email:</label><br>

        <input type="text" id="email"><br><br>
```

```
<label>Phone Number:</label><br>
<input type="text" id="phone"><br><br>

<input type="submit" value="Register">
</form>

<p id="error" style="color:red;"></p>

</body>
</html>
```

Step 6: Create JavaScript Validation File

static/js/validate.js

```
function validateForm() {
    var email = document.getElementById("email").value;
    var phone = document.getElementById("phone").value;
    var error = document.getElementById("error");

    var emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    var phonePattern = /^[0-9]{10}$/;

    if (email === "" || phone === "") {
        error.innerHTML = "All fields are required!";
        return false;
    }

    if (!emailPattern.test(email)) {
```

```

        error.innerHTML = "Invalid email format!";
        return false;
    }

    if (!phonePattern.test(phone)) {
        error.innerHTML = "Phone number must be 10 digits!";
        return false;
    }

    alert("Registration successful!");
    return true;
}

```

Step 7: Create View

accounts/views.py

```
from django.shortcuts import render
```

```
def register(request):
    return render(request, 'register.html')
```

Step 8: Configure URLs

App URLs (accounts/urls.py)

```
from django.urls import path
```

```
from . import views
```

```
urlpatterns = [
    path("", views.register, name='register'),
]

```

Project URLs (regvalidation/urls.py)

```
from django.contrib import admin
from django.urls import path, include
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path("", include('accounts.urls')),
]
```

Step 9: Run the Server

```
python manage.py runserver
```

11. Django Database Connectivity (MySQL or SQLite)

Q.11 Set up database connectivity for a Django project.

Ans. 1.Database: SQLite

Step 1: Check Database Settings

settings.py:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

Step 2: Apply Migrations

```
python manage.py makemigrations
```

```
python manage.py migrate
```

Step 3: Test Database Connectivity

```
python manage.py runserver
```

2. Using MySQL Database

Step 1: Install MySQL Connector

```
pip install mysqlclient
```

Step 2: Create Database in MySQL

```
CREATE DATABASE doctorfinder_db;
```

Step 3: Configure MySQL in settings.py

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.mysql',  
        'NAME': 'doctorfinder_db',  
        'USER': 'root',  
        'PASSWORD': 'password',  
        'HOST': 'localhost',  
        'PORT': '3306',  
    }  
}
```

Step 4: Apply Migrations

```
python manage.py makemigrations
```

```
python manage.py migrate
```

Step 5: Verify Database Connection

```
python manage.py runserver
```

Practical Example:

11) Write a Django project to connect to an SQLite/MySQL database and manage doctor records.

Ans.

Step 1: Create Django Project and App

```
django-admin startproject doctor_db
```

```
cd doctor_db
```

```
python manage.py startapp doctors
```

Step 2: Register App in settings.py

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'doctors',  
]
```

Step 3: Database Configuration

Option A: SQLite

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.sqlite3',  
        'NAME': BASE_DIR / 'db.sqlite3',  
    }  
}
```

Option B: MySQL

Install connector:

```
pip install mysqlclient
```

Create database:

```
CREATE DATABASE doctor_db;
```

Configure settings.py:

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.mysql',  
        'NAME': 'doctor_db',  
        'USER': 'root',  
        'PASSWORD': 'password',  
        'HOST': 'localhost',  
        'PORT': '3306',  
    }  
}
```

Step 4: Create Doctor Model

doctors/models.py

```
from django.db import models  
  
class Doctor(models.Model):  
    name = models.CharField(max_length=100)  
    specialization = models.CharField(max_length=100)  
    experience = models.IntegerField()  
    phone = models.CharField(max_length=10)  
  
    def __str__(self):  
        return self.name
```

Step 5: Apply Migrations

```
python manage.py makemigrations
```

```
python manage.py migrate
```

Step 6: Register Doctor Model in Admin

doctors/admin.py

```
from django.contrib import admin
```

```
from .models import Doctor
```

```
admin.site.register(Doctor)
```

Create admin user:

```
python manage.py createsuperuser
```

Step 7: Create View to Display Doctors

doctors/views.py

```
from django.shortcuts import render
```

```
from .models import Doctor
```

```
def doctor_list(request):
```

```
    doctors = Doctor.objects.all()
```

```
    return render(request, 'doctor_list.html', {'doctors': doctors})
```

Step 8: Create Template

Create folder:

doctors/templates/doctor_list.html

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <title>Doctor Records</title>
```

```
</head>
```

```

<body>

<h1>Doctor Records</h1>

{% for doctor in doctors %}

    <p>
        <b>Name:</b> {{ doctor.name }} <br>
        <b>Specialization:</b> {{ doctor.specialization }} <br>
        <b>Experience:</b> {{ doctor.experience }} years <br>
        <b>Phone:</b> {{ doctor.phone }}
    </p>
    <hr>

{% empty %}
    <p>No doctors available.</p>
{% endfor %}

</body>
</html>

```

Step 9: Configure URLs

App URLs (doctors/urls.py)

```

from django.urls import path
from . import views

urlpatterns = [
    path("", views.doctor_list, name='doctor_list'),
]

```

Project URLs (doctor_db/urls.py)

```
from django.contrib import admin
from django.urls import path, include
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path("", include('doctors.urls')),
]
```

Step 10: Run Server

```
python manage.py runserver
```

12. ORM and QuerySets

Q.12 Perform CRUD operations using Django ORM.

Ans. CRUD Operations Using Django ORM

CRUD stands for:

- Create
- Read
- Update
- Delete

Step 1: Doctor Model

models.py

```
from django.db import models
```

```
class Doctor(models.Model):
```

```
    name = models.CharField(max_length=100)
```

```
    specialization = models.CharField(max_length=100)
```

```
    experience = models.IntegerField()
```

```
phone = models.CharField(max_length=10)

def __str__(self):
    return self.name
```

Run migrations:

```
python manage.py makemigrations
```

```
python manage.py migrate
```

1 CREATE (Insert Data)

Method 1: Using save()

```
doctor = Doctor(
    name="Dr. Rahul Verma",
    specialization="Dentist",
    experience=5,
    phone="9876543210"
)
```

```
doctor.save()
```

Method 2: Using create()

```
Doctor.objects.create(
    name="Dr. Anjali Singh",
    specialization="Pediatrician",
    experience=8,
    phone="9123456789"
)
```

2 READ (Retrieve Data)

Get all doctors

```
doctors = Doctor.objects.all()
```

Get single doctor

```
doctor = Doctor.objects.get(id=1)
```

Filter doctors

```
doctors = Doctor.objects.filter(specialization="Dentist")
```

Get first / last record

```
Doctor.objects.first()
```

```
Doctor.objects.last()
```

3 UPDATE (Modify Data)

Update using save()

```
doctor = Doctor.objects.get(id=1)
```

```
doctor.experience = 10
```

```
doctor.save()
```

Update using update()

```
Doctor.objects.filter(id=2).update(phone="9999999999")
```

4 DELETE (Remove Data)

Delete a single record

```
doctor = Doctor.objects.get(id=1)
```

```
doctor.delete()
```

Delete multiple records

```
Doctor.objects.filter(specialization="Dentist").delete()
```

CRUD Using Django Shell

Open shell:

```
python manage.py shell
```

Then:

```
from doctors.models import Doctor
```

```
Doctor.objects.all()
```

Practical Example:

12) Write a Django project that demonstrates CRUD operations (Create, Read, Update, Delete) on doctor profiles using Django ORM.

Ans.

1 Create Django Project and App

```
django-admin startproject doctor_crud
```

```
cd doctor_crud
```

```
python manage.py startapp doctors
```

2 Register App in settings.py

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'doctors',  
]
```

3 Create Doctor Model

doctors/models.py

```
from django.db import models
```

```
class Doctor(models.Model):  
    name = models.CharField(max_length=100)  
    specialization = models.CharField(max_length=100)  
    experience = models.IntegerField()
```

```
phone = models.CharField(max_length=10)
```

```
def __str__(self):  
    return self.name
```

Apply migrations:

```
python manage.py makemigrations
```

```
python manage.py migrate
```

4 Register Model in Admin Panel

doctors/admin.py

```
from django.contrib import admin
```

```
from .models import Doctor
```

```
admin.site.register(Doctor)
```

5 Create Views (CRUD Operations)

doctors/views.py

```
from django.shortcuts import render, redirect, get_object_or_404
```

```
from .models import Doctor
```

```
# READ#
```

```
def doctor_list(request):
```

```
    doctors = Doctor.objects.all()
```

```
    return render(request, 'doctor_list.html', {'doctors': doctors})
```

```
# CREATE#
```

```
def add_doctor(request):
```

```
if request.method == 'POST':
    Doctor.objects.create(
        name=request.POST['name'],
        specialization=request.POST['specialization'],
        experience=request.POST['experience'],
        phone=request.POST['phone']
    )
    return redirect('doctor_list')
return render(request, 'add_doctor.html')
```

UPDATE#

```
def update_doctor(request, id):
    doctor = get_object_or_404(Doctor, id=id)
    if request.method == 'POST':
        doctor.name = request.POST['name']
        doctor.specialization = request.POST['specialization']
        doctor.experience = request.POST['experience']
        doctor.phone = request.POST['phone']
        doctor.save()
        return redirect('doctor_list')
    return render(request, 'update_doctor.html', {'doctor': doctor})
```

DELETE#

```
def delete_doctor(request, id):
    doctor = get_object_or_404(Doctor, id=id)
    doctor.delete()
    return redirect('doctor_list')
```

6 URL Configuration

App URLs – `doctors/urls.py`

```
from django.urls import path
from . import views

urlpatterns = [
    path("", views.doctor_list, name='doctor_list'),
    path('add/', views.add_doctor, name='add_doctor'),
    path('update/<int:id>/', views.update_doctor, name='update_doctor'),
    path('delete/<int:id>/', views.delete_doctor, name='delete_doctor'),
]
```

Project URLs – `doctor_crud/urls.py`

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path("", include('doctors.urls')),
]
```

7 Templates (HTML)

`doctor_list.html`

```
<h2>Doctor Profiles</h2>
<a href="/add/">Add Doctor</a>

{% for d in doctors %}

<p>
```

```
    {{ d.name }} - {{ d.specialization }} - {{ d.experience }} years - {{ d.phone }}
    <a href="/update/{{ d.id }}">Edit</a>
    <a href="/delete/{{ d.id }}">Delete</a>
</p>
{% endfor %}
```

add_doctor.html

```
<h2>Add Doctor</h2>
<form method="post">
    {% csrf_token %}
    Name: <input type="text" name="name"><br>
    Specialization: <input type="text" name="specialization"><br>
    Experience: <input type="number" name="experience"><br>
    Phone: <input type="text" name="phone"><br>
    <button type="submit">Save</button>
</form>
```

update_doctor.html

```
<h2>Update Doctor</h2>
<form method="post">
    {% csrf_token %}
    Name: <input type="text" name="name" value="{{ doctor.name }}"><br>
    Specialization: <input type="text" name="specialization" value="{{ doctor.specialization }}"><br>
    Experience: <input type="number" name="experience" value="{{ doctor.experience }}"><br>
    Phone: <input type="text" name="phone" value="{{ doctor.phone }}"><br>
```

```
<button type="submit">Update</button>
</form>
```

8 Run Server

```
python manage.py runserver
```

13. Django Forms and Authentication

Q.13 Create a Django project for user registration and login functionality.

Ans.

1 Create Django Project and App

```
django-admin startproject user_auth
```

```
cd user_auth
```

```
python manage.py startapp accounts
```

2 Register App in settings.py

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'accounts',
]
```

Add login redirects:

```
LOGIN_REDIRECT_URL = 'dashboard'
```

```
LOGOUT_REDIRECT_URL = 'login'
```

3 Create Views (Registration, Login, Logout)

accounts/views.py

```
from django.shortcuts import render, redirect
from django.contrib.auth.forms import UserCreationForm, AuthenticationForm
from django.contrib.auth import login, logout
```

```
# User Registration#
```

```
def register(request):
    if request.method == 'POST':
        form = UserCreationForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect('login')
    else:
        form = UserCreationForm()
    return render(request, 'register.html', {'form': form})
```

```
# User Login#
```

```
def user_login(request):
    if request.method == 'POST':
        form = AuthenticationForm(data=request.POST)
        if form.is_valid():
            login(request, form.get_user())
            return redirect('dashboard')
    else:
        form = AuthenticationForm()
    return render(request, 'login.html', {'form': form})
```

```
# User Logout#
def user_logout(request):
    logout(request)
    return redirect('login')

# Dashboard Page#
def dashboard(request):
    return render(request, 'dashboard.html')
```

4 URL Configuration

App URLs - accounts/urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('register/', views.register, name='register'),
    path('login/', views.user_login, name='login'),
    path('logout/', views.user_logout, name='logout'),
    path('dashboard/', views.dashboard, name='dashboard'),
]
```

Project URLs – user_auth/urls.py

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
```

```
    path('admin/', admin.site.urls),
    path("", include('accounts.urls')),
]
```

5 Templates

Create folder:

accounts/templates/

register.html

```
<h2>User Registration</h2>
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Register</button>
</form>
<a href="/login/">Already have an account?</a>
```

login.html

```
<h2>User Login</h2>
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Login</button>
</form>
<a href="/register/">Create new account</a>
```

dashboard.html

```
<h2>Welcome, {{ user.username }}</h2>
<p>You are successfully logged in.</p>
<a href="/logout/">Logout</a>
```

6 Apply Migrations

```
python manage.py migrate
```

7 Create Superuser (Opt)

```
python manage.py createsuperuser
```

8 Run Server

```
python manage.py runserver
```

Practical Example:

13) Write a Django project to handle user sign up, login, password reset, and profile updates.

Ans.

1 Create Project and App

```
django-admin startproject auth_system
```

```
cd auth_system
```

```
python manage.py startapp accounts
```

2 Register App & Auth Settings (settings.py)

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
```

```
'django.contrib.sessions',  
'django.contrib.messages',  
'django.contrib.staticfiles',  
'accounts',  
]
```

```
LOGIN_REDIRECT_URL = 'profile'  
LOGOUT_REDIRECT_URL = 'login'
```

```
EMAIL_BACKEND = 'django.core.mail.backends.console.EmailBackend'
```

3 Create Profile Model

accounts/models.py

```
from django.db import models  
from django.contrib.auth.models import User
```

```
class Profile(models.Model):  
    user = models.OneToOneField(User, on_delete=models.CASCADE)  
    phone = models.CharField(max_length=10)  
    address = models.TextField(blank=True)  
  
    def __str__(self):  
        return self.user.username
```

4 Signals to Auto-Create Profile

accounts/signals.py

```
from django.db.models.signals import post_save
```

```
from django.dispatch import receiver
from django.contrib.auth.models import User
from .models import Profile

@receiver(post_save, sender=User)
def create_profile(sender, instance, created, **kwargs):
    if created:
        Profile.objects.create(user=instance)
```

accounts/apps.py

```
from django.apps import AppConfig

class AccountsConfig(AppConfig):
    name = 'accounts'

    def ready(self):
        import accounts.signals
```

5 Migrations

```
python manage.py makemigrations
python manage.py migrate
```

6 Views (Signup, Login, Profile Update)

accounts/views.py

```
from django.shortcuts import render, redirect
from django.contrib.auth.forms import UserCreationForm
from django.contrib.auth.decorators import login_required
```

```
from .models import Profile
```

```
# Signup#
```

```
def signup(request):
```

```
    if request.method == 'POST':
```

```
        form = UserCreationForm(request.POST)
```

```
        if form.is_valid():
```

```
            form.save()
```

```
            return redirect('login')
```

```
    else:
```

```
        form = UserCreationForm()
```

```
    return render(request, 'signup.html', {'form': form})
```

```
# Profile#
```

```
@login_required
```

```
def profile(request):
```

```
    profile = request.user.profile
```

```
    if request.method == 'POST':
```

```
        profile.phone = request.POST['phone']
```

```
        profile.address = request.POST['address']
```

```
        profile.save()
```

```
    return render(request, 'profile.html', {'profile': profile})
```

7 URLs Configuration

App URLs (accounts/urls.py)

```
from django.urls import path
from django.contrib.auth import views as auth_views
from . import views

urlpatterns = [
    path('signup/', views.signup, name='signup'),
    path('profile/', views.profile, name='profile'),

    path('login/', auth_views.LoginView.as_view(template_name='login.html'),
name='login'),
    path('logout/', auth_views.LogoutView.as_view(), name='logout'),

    path('password_reset/',
auth_views.PasswordResetView.as_view(template_name='password_reset.html'
), name='password_reset'),
    path('password_reset_done/',
auth_views.PasswordResetDoneView.as_view(), name='password_reset_done'),
    path('reset/<uidb64>/<token>',
auth_views.PasswordResetConfirmView.as_view(),
name='password_reset_confirm'),
    path('reset_done/', auth_views.PasswordResetCompleteView.as_view(),
name='password_reset_complete'),
]
```

Project URLs (auth_system/urls.py)

```
from django.contrib import admin
from django.urls import path, include
```

```
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path("", include('accounts.urls')),  
]
```

8 Templates (HTML)

signup.html

```
<h2>Sign Up</h2>  
<form method="post">  
    {% csrf_token %}  
    {{ form.as_p }}  
    <button>Register</button>  
</form>
```

login.html

```
<h2>Login</h2>  
<form method="post">  
    {% csrf_token %}  
    {{ form.as_p }}  
    <button>Login</button>  
</form>  
<a href="/password_reset/">Forgot Password?</a>
```

profile.html

```
<h2>Profile</h2>  
<form method="post">  
    {% csrf_token %}
```

```
Phone: <input type="text" name="phone" value="{{ profile.phone }}"><br>
Address: <textarea name="address">{{ profile.address }}</textarea><br>
<button>Update</button>
</form>
<a href="/logout/">Logout</a>
```

password_reset.html

```
<h2>Password Reset</h2>
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button>Send Reset Link</button>
</form>
```

9 Run Server

```
python manage.py runserver
```

14. CRUD Operations using AJAX

Q.14 Implement AJAX in a Django project for performing CRUD operations.

Ans.

1 Create Project and App

```
django-admin startproject ajax_crud
```

```
cd ajax_crud
```

```
python manage.py startapp doctors
```

2 Register App (settings.py)

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'doctors',  
]
```

3 Create Model

doctors/models.py

```
from django.db import models
```

```
class Doctor(models.Model):  
    name = models.CharField(max_length=100)  
    specialization = models.CharField(max_length=100)  
    experience = models.IntegerField()  
  
    def __str__(self):  
        return self.name
```

Apply migrations:

```
python manage.py makemigrations
```

```
python manage.py migrate
```

4 Views for AJAX CRUD

doctors/views.py

```
from django.http import JsonResponse
from django.shortcuts import render, get_object_or_404
from .models import Doctor
```

```
def index(request):
    return render(request, 'index.html')
```

READ#

```
def doctor_list(request):
    doctors = list(Doctor.objects.values())
    return JsonResponse(doctors, safe=False)
```

CREATE#

```
def add_doctor(request):
    if request.method == 'POST':
        Doctor.objects.create(
            name=request.POST['name'],
            specialization=request.POST['specialization'],
            experience=request.POST['experience']
        )
    return JsonResponse({'status': 'Doctor Added'})
```

UPDATE#

```
def update_doctor(request, id):
    doctor = get_object_or_404(Doctor, id=id)
```

```

if request.method == 'POST':
    doctor.name = request.POST['name']
    doctor.specialization = request.POST['specialization']
    doctor.experience = request.POST['experience']
    doctor.save()
    return JsonResponse({'status': 'Doctor Updated'})

```

DELETE#

```

def delete_doctor(request, id):
    doctor = get_object_or_404(Doctor, id=id)
    doctor.delete()
    return JsonResponse({'status': 'Doctor Deleted'})

```

5 URL Configuration

App URLs (doctors/urls.py)

```

from django.urls import path
from . import views

```

```

urlpatterns = [
    path("", views.index),
    path('list/', views.doctor_list),
    path('add/', views.add_doctor),
    path('update/<int:id>', views.update_doctor),
    path('delete/<int:id>', views.delete_doctor),
]

```

Project URLs (ajax_crud/urls.py)

```
from django.contrib import admin
from django.urls import path, include
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path("", include('doctors.urls')),
]
```

6 Template with AJAX (JavaScript)

doctors/templates/index.html

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <title>AJAX CRUD</title>
```

```
</head>
```

```
<body>
```

```
<h2>Doctor Management (AJAX)</h2>
```

```
<form id="doctorForm">
```

```
    Name: <input type="text" id="name">
```

```
    Specialization: <input type="text" id="specialization">
```

```
    Experience: <input type="number" id="experience">
```

```
    <button type="submit">Add Doctor</button>
```

```
</form>
```

```
<hr>
```

```
<div id="doctorList"></div>
```

```
<script>
```

```
function loadDoctors() {
```

```
  fetch('/list/')
```

```
  .then(res => res.json())
```

```
  .then(data => {
```

```
    let html = "";
```

```
    data.forEach(d => {
```

```
      html += `
```

```
        <p>
```

```
          ${d.name} - ${d.specialization} - ${d.experience} years
```

```
          <button onclick="deleteDoctor(${d.id})">Delete</button>
```

```
        </p>`;
```

```
    });
```

```
    document.getElementById('doctorList').innerHTML = html;
```

```
  });
```

```
}
```

```
document.getElementById('doctorForm').addEventListener('submit',
```

```
function(e){
```

```
  e.preventDefault();
```

```
  fetch('/add/', {
```

```
    method: 'POST',
```

```
    headers: {
```

```
      'X-CSRFToken': '{{ csrf_token }}'
```

```
    },
```

```
        body: new URLSearchParams({
            name: name.value,
            specialization: specialization.value,
            experience: experience.value
        })
    }).then(() => loadDoctors());
});
```

```
function deleteDoctor(id) {
    fetch(`/delete/${id}/`)
    .then(() => loadDoctors());
}
```

```
loadDoctors();
```

```
</script>
```

```
</body>
```

```
</html>
```

7 Run Server

```
python manage.py runserver
```

Practical Example:

14) Write a Django project that uses AJAX to add, edit, or delete doctor profiles without refreshing the page.

Ans.

1 Create Project and App

```
django-admin startproject doctor_ajax
```

```
cd doctor_ajax
```

```
python manage.py startapp doctors
```

2 Register App (settings.py)

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'doctors',  
]
```

3 Create Doctor Model

doctors/models.py

```
from django.db import models
```

```
class Doctor(models.Model):
```

```
    name = models.CharField(max_length=100)
```

```
    specialization = models.CharField(max_length=100)
```

```
    experience = models.IntegerField()
```

```
    def __str__(self):
```

```
        return self.name
```

Run migrations:

```
python manage.py makemigrations
```

```
python manage.py migrate
```

4 Views (AJAX CRUD)

doctors/views.py

```
from django.shortcuts import render, get_object_or_404
```

```
from django.http import JsonResponse
```

```
from .models import Doctor
```

```
def index(request):
```

```
    return render(request, 'index.html')
```

```
# READ#
```

```
def doctor_list(request):
```

```
    data = list(Doctor.objects.values())
```

```
    return JsonResponse(data, safe=False)
```

```
# CREATE#
```

```
def add_doctor(request):
```

```
    if request.method == 'POST':
```

```
        Doctor.objects.create(
```

```
            name=request.POST['name'],
```

```
            specialization=request.POST['specialization'],
```

```
            experience=request.POST['experience']
```

```
        )
```

```
    return JsonResponse({'status': 'Doctor Added'})
```

```
# UPDATE#
```

```
def update_doctor(request, id):  
    doctor = get_object_or_404(Doctor, id=id)  
    if request.method == 'POST':  
        doctor.name = request.POST['name']  
        doctor.specialization = request.POST['specialization']  
        doctor.experience = request.POST['experience']  
        doctor.save()  
        return JsonResponse({'status': 'Doctor Updated'})
```

```
# DELETE#
```

```
def delete_doctor(request, id):  
    doctor = get_object_or_404(Doctor, id=id)  
    doctor.delete()  
    return JsonResponse({'status': 'Doctor Deleted'})
```

5 URL Configuration

App URLs – doctors/urls.py

```
from django.urls import path
```

```
from . import views
```

```
urlpatterns = [  
    path("", views.index),  
    path('list/', views.doctor_list),  
    path('add/', views.add_doctor),  
    path('update/<int:id>/', views.update_doctor),
```

```
    path('delete/<int:id>/', views.delete_doctor),  
]
```

Project URLs – doctor_ajax/urls.py

```
from django.contrib import admin  
from django.urls import path, include
```

```
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path("", include('doctors.urls')),  
]
```

6 Template with AJAX

doctors/templates/index.html

```
<!DOCTYPE html>  
<html>  
<head>  
    <title>Doctor AJAX CRUD</title>  
</head>  
<body>  
  
    <h2>Doctor Profiles (AJAX)</h2>  
  
    <form id="doctorForm">  
        {% csrf_token %}  
        <input type="hidden" id="doctor_id">  
        Name: <input type="text" id="name">
```

```

    Specialization: <input type="text" id="specialization">
    Experience: <input type="number" id="experience">
    <button type="submit">Save</button>
</form>

<hr>

<div id="doctorList"></div>

<script>
function loadDoctors() {
    fetch('/list/')
    .then(res => res.json())
    .then(data => {
        let html = "";
        data.forEach(d => {
            html += `
                <p>
                    ${d.name} - ${d.specialization} - ${d.experience} years
                    <button onclick="editDoctor(${d.id}, '${d.name}',
                    '${d.specialization}', ${d.experience})">Edit</button>
                    <button onclick="deleteDoctor(${d.id})">Delete</button>
                </p>`;
        });
        document.getElementById('doctorList').innerHTML = html;
    });
}

```

```

document.getElementById('doctorForm').addEventListener('submit',
function(e){
    e.preventDefault();
    let id = doctor_id.value;
    let url = id ? `/update/${id}/` : '/add/';

    fetch(url, {
        method: 'POST',
        headers: {'X-CSRFToken':
document.querySelector('[name=csrfmiddlewaretoken]').value},
        body: new URLSearchParams({
            name: name.value,
            specialization: specialization.value,
            experience: experience.value
        })
    }).then(() => {
        doctorForm.reset();
        doctor_id.value = "";
        loadDoctors();
    });
});

```

```

function editDoctor(id, nameVal, spec, exp) {
    doctor_id.value = id;
    name.value = nameVal;
    specialization.value = spec;
    experience.value = exp;
}

```

```
function deleteDoctor(id) {  
    fetch(`/delete/${id}/`)  
    .then(() => loadDoctors());  
}
```

```
loadDoctors();
```

```
</script>
```

```
</body>
```

```
</html>
```

7 Run Server

```
python manage.py runserver
```

15. Customizing the Django Admin Panel

Q.15 Customize the Django admin panel for better management of records.

Ans.

1 Example Model

models.py

```
from django.db import models
```

```
class Doctor(models.Model):
```

```
    name = models.CharField(max_length=100)
```

```
    specialization = models.CharField(max_length=100)
```

```
    experience = models.IntegerField()
```

```
    phone = models.CharField(max_length=10)
```

```
is_available = models.BooleanField(default=True)
```

```
def __str__(self):  
    return self.name
```

2 Register Model with Custom Admin

admin.py

```
from django.contrib import admin
```

```
from .models import Doctor
```

```
@admin.register(Doctor)
```

```
class DoctorAdmin(admin.ModelAdmin):
```

```
    list_display = ('name', 'specialization', 'experience', 'phone', 'is_available')
```

```
    list_filter = ('specialization', 'is_available')
```

```
    search_fields = ('name', 'specialization', 'phone')
```

```
    list_per_page = 10
```

```
    list_editable = ('is_available',)
```

```
    ordering = ('name',)
```

3 Field Grouping (Fieldsets)

```
@admin.register(Doctor)

class DoctorAdmin(admin.ModelAdmin):

    fieldsets = (

        ('Personal Info', {

            'fields': ('name', 'phone')

        }),

        ('Professional Details', {

            'fields': ('specialization', 'experience')

        }),

        ('Status', {

            'fields': ('is_available',)

        }),

    )
```

4 Read-Only Fields

```
class DoctorAdmin(admin.ModelAdmin):

    readonly_fields = ('experience',)
```

5 Custom Actions

```
class DoctorAdmin(admin.ModelAdmin):

    actions = ['make_available']

    def make_available(self, request, queryset):

        queryset.update(is_available=True)

    make_available.short_description = "Mark selected doctors as available"
```

6 Inline Models (Related Data)

```
class Appointment(models.Model):  
    doctor = models.ForeignKey(Doctor, on_delete=models.CASCADE)  
    patient_name = models.CharField(max_length=100)
```

```
class AppointmentInline(admin.TabularInline):  
    model = Appointment  
    extra = 1
```

```
class DoctorAdmin(admin.ModelAdmin):  
    inlines = [AppointmentInline]
```

7 Custom Admin Site Title

admin.py

```
admin.site.site_header = "Doctor Finder Admin"  
admin.site.site_title = "Doctor Finder Portal"  
admin.site.index_title = "Manage Doctor Records"
```

Practical Example:

15) Write a Django project that customizes the admin panel to display more detailed doctor information (e.g., specialties, availability).

Ans.

1 Create Project and App

```
django-admin startproject doctor_admin  
cd doctor_admin  
python manage.py startapp doctors
```

2 Register App (settings.py)

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'doctors',  
]
```

3 Create Doctor Model

doctors/models.py

```
from django.db import models  
  
class Doctor(models.Model):  
    name = models.CharField(max_length=100)  
    specialty = models.CharField(max_length=100)  
    experience = models.IntegerField()  
    phone = models.CharField(max_length=10)  
    available = models.BooleanField(default=True)  
  
    def __str__(self):  
        return self.name
```

4 Apply Migrations

```
python manage.py makemigrations
```

```
python manage.py migrate
```

5 Customize Admin Panel

doctors/admin.py

```
from django.contrib import admin
```

```
from .models import Doctor
```

```
@admin.register(Doctor)
```

```
class DoctorAdmin(admin.ModelAdmin):
```

```
    list_display = ('name', 'specialty', 'experience', 'phone', 'available')
```

```
    list_filter = ('specialty', 'available')
```

```
    search_fields = ('name', 'specialty', 'phone')
```

```
    list_editable = ('available',)
```

```
    list_per_page = 10
```

```
    ordering = ('name',)
```

```
    fieldsets = (
        ('Doctor Information', {
            'fields': ('name', 'phone')
        }),
        ('Professional Details', {
            'fields': ('specialty', 'experience')
        }),
    ),
```

```
('Availability Status', {  
    'fields': ('available',)  
}),  
)
```

6 Customize Admin Titles (Opt)

doctors/admin.py

```
admin.site.site_header = "Doctor Finder Admin Panel"  
admin.site.site_title = "Doctor Finder Admin"  
admin.site.index_title = "Manage Doctor Information"
```

7 Create Superuser

```
python manage.py createsuperuser
```

8 Run Server

```
python manage.py runserver
```

16. Payment Integration Using Paytm

Q.16 Implement Paytm payment gateway in a Django project.

Ans.

1 Create Django Project and App

```
django-admin startproject paytm_project
```

```
cd paytm_project
```

```
python manage.py startapp payments
```

2 Register App (settings.py)

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'payments',  
]
```

3 Paytm Configuration (Settings)

settings.py

```
PAYTM_MERCHANT_ID = "YOUR_MERCHANT_ID"
```

```
PAYTM_MERCHANT_KEY = "YOUR_MERCHANT_KEY"
```

```
PAYTM_WEBSITE = "WEBSTAGING"
```

```
PAYTM_INDUSTRY_TYPE = "Retail"
```

```
PAYTM_CHANNEL_ID = "WEB"
```

```
PAYTM_CALLBACK_URL = http://127.0.0.1:8000/payment/response/
```

4 Payment Model (Opt)

payments/models.py

```
from django.db import models

class Payment(models.Model):
    order_id = models.CharField(max_length=50)
    amount = models.FloatField()
    status = models.CharField(max_length=20)

    def __str__(self):
        return self.order_id
```

Run:

```
python manage.py makemigrations
```

```
python manage.py migrate
```

5 Paytm Checksum Utility

Create file:

```
payments/paytm_checksum.py
```

6 Views for Payment

payments/views.py

```
from django.shortcuts import render
from django.conf import settings
import uuid
```

```
def initiate_payment(request):
    order_id = str(uuid.uuid4())
```

```

param_dict = {
    'MID': settings.PAYTM_MERCHANT_ID,
    'ORDER_ID': order_id,
    'TXN_AMOUNT': '500',
    'CUST_ID': 'cust001',
    'INDUSTRY_TYPE_ID': settings.PAYTM_INDUSTRY_TYPE,
    'WEBSITE': settings.PAYTM_WEBSITE,
    'CHANNEL_ID': settings.PAYTM_CHANNEL_ID,
    'CALLBACK_URL': settings.PAYTM_CALLBACK_URL,
}

```

```

return render(request, 'paytm.html', {'param_dict': param_dict})

```

```

def payment_response(request):
    response = request.POST.dict()
    return render(request, 'payment_status.html', {'response': response})

```

7 URLs Configuration

App URLs – payments/urls.py

```

from django.urls import path

```

```

from . import views

```

```

urlpatterns = [
    path('pay/', views.initiate_payment, name='pay'),
    path('response/', views.payment_response, name='response'),
]

```

Project URLs – paytm_project/urls.py

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('payment/', include('payments.urls')),
]
```

8 Payment Template

templates/paytm.html

```
<h2>Redirecting to Paytm...</h2>
```

```
<form method="post" action="https://securegw-stage.paytm.in/order/process">
    {% for key, value in param_dict.items %}
        <input type="hidden" name="{{ key }}" value="{{ value }}">
    {% endfor %}
    <button type="submit">Pay Now</button>
</form>
```

9 Payment Status Page

templates/payment_status.html

```
<h2>Payment Status</h2>
```

```
{% for key, value in response.items %}
    <p>{{ key }} : {{ value }}</p>
{% endfor %}
```

10 Run Server

python manage.py runserver

Open:

<http://127.0.0.1:8000/payment/pay/>

Practical Example:

16) Write a Django project that integrates Paytm for handling payments in the "Doctor Finder" project.

Ans.

1 Create Project & Apps

```
django-admin startproject doctor_finder
```

```
cd doctor_finder
```

```
python manage.py startapp doctors
```

```
python manage.py startapp payments
```

2 Register Apps (settings.py)

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'doctors',  
    'payments',  
]
```

3 Paytm Configuration (Sandbox)

settings.py

```
PAYTM_MERCHANT_ID = "YOUR_MERCHANT_ID"  
PAYTM_MERCHANT_KEY = "YOUR_MERCHANT_KEY"  
PAYTM_WEBSITE = "WEBSTAGING"  
PAYTM_INDUSTRY_TYPE = "Retail"  
PAYTM_CHANNEL_ID = "WEB"  
PAYTM_CALLBACK_URL = http://127.0.0.1:8000/payment/response/
```

4 Doctor Model

doctors/models.py

```
from django.db import models  
  
class Doctor(models.Model):  
    name = models.CharField(max_length=100)  
    specialization = models.CharField(max_length=100)  
    consultation_fee = models.IntegerField()  
  
    def __str__(self):  
        return self.name
```

5 Payment Model

payments/models.py

```
from django.db import models  
  
class Payment(models.Model):  
    order_id = models.CharField(max_length=50)
```

```
doctor = models.CharField(max_length=100)
amount = models.IntegerField()
status = models.CharField(max_length=20)
```

```
def __str__(self):
    return self.order_id
```

Run migrations:

```
python manage.py makemigrations
python manage.py migrate
```

6 Payment Views

payments/views.py

```
from django.shortcuts import render
from django.conf import settings
from .models import Payment
import uuid
```

```
def initiate_payment(request, doctor_name, fee):
    order_id = str(uuid.uuid4())
```

```
    paytm_params = {
        'MID': settings.PAYTM_MERCHANT_ID,
        'ORDER_ID': order_id,
        'TXN_AMOUNT': str(fee),
        'CUST_ID': 'user001',
        'INDUSTRY_TYPE_ID': settings.PAYTM_INDUSTRY_TYPE,
        'WEBSITE': settings.PAYTM_WEBSITE,
```

```
'CHANNEL_ID': settings.PAYTM_CHANNEL_ID,  
'CALLBACK_URL': settings.PAYTM_CALLBACK_URL,  
}
```

```
Payment.objects.create(  
    order_id=order_id,  
    doctor=doctor_name,  
    amount=fee,  
    status='INITIATED'  
)
```

```
return render(request, 'paytm.html', {'params': paytm_params})
```

```
def payment_response(request):  
    response = request.POST.dict()  
    order_id = response.get('ORDERID')  
    status = response.get('STATUS', 'FAILED')
```

```
Payment.objects.filter(order_id=order_id).update(status=status)
```

```
return render(request, 'payment_status.html', {'response': response})
```

7 URLs Configuration

payments/urls.py

```
from django.urls import path
```

```
from . import views
```

```
urlpatterns = [  
    path('pay/<str:doctor_name>/<int:fee>/', views.initiate_payment),  
    path('response/', views.payment_response),  
]
```

doctor_finder/urls.py

```
from django.contrib import admin
```

```
from django.urls import path, include
```

```
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path('payment/', include('payments.urls')),  
]
```

8 Paytm Redirect Template

templates/paytm.html

```
<h2>Redirecting to Paytm...</h2>
```

```
<form method="post" action="https://securegw-stage.paytm.in/order/process">
```

```
    {% for key, value in params.items %}
```

```
        <input type="hidden" name="{{ key }}" value="{{ value }}">
```

```
    {% endfor %}
```

```
    <button type="submit">Pay Now</button>
```

```
</form>
```

9 Payment Status Page

templates/payment_status.html

```
<h2>Payment Status</h2>
```

```
{% for key, value in response.items %}
```

```
    <p>{{ key }} : {{ value }}</p>
```

```
{% endfor %}
```

```
<a href="/">Back to Doctor Finder</a>
```

10 Run Server

```
python manage.py runserver
```

Open:

<http://127.0.0.1:8000/payment/pay/DrAmit/500/>

17. GitHub Project Deployment

Q.17 Deploy a Django project to GitHub for version control.

Ans.

1 Prerequisites

Git installed

GitHub account

Django project already created

Check Git:

```
git --version
```

2 Go to Your Django Project Folder

Example:

```
cd doctor_finder
```

project structure:

doctor_finder/

├── manage.py

├── doctor_finder/

├── doctors/

└── payments/

3 Initialize Git Repository

git init

4 Create .gitignore File

Create a file named .gitignore in the root folder.

.gitignore

__pycache__/

*.pyc

db.sqlite3

.env

venv/

myenv/

5 Check Git Status

git status

6 Add Files to Git

git add .

7 Commit Changes

git commit -m "Initial Django project commit"

8 Create Repository on GitHub

1. Go to <https://github.com>
2. Click New Repository
3. Repository name:
doctor-finder-django
4. Keep it Public
5. Do NOT add README (already local)

9 Connect Local Project to GitHub

Copy repository URL and run:

```
git remote add origin https://github.com/username/doctor-finder-django.git
```

Verify:

```
git remote -v
```

10 Push Django Project to GitHub

```
git branch -M main
```

```
git push -u origin main
```

Practical Example:

17) Write a step-by-step guide to deploying the “Doctor Finder” project to GitHub.

Ans.

Step 1: Create the Django Project

```
django-admin startproject doctor_finder
```

```
cd doctor_finder
```

Project structure:

doctor_finder/

├── manage.py

├── doctor_finder/

└── doctors/

Step 2: Create and Activate Virtual Environment

```
python -m venv venv
```

Activate:

```
venv\Scripts\activate
```

Step 3: Install Django

```
pip install Django
```

Step 4: Initialize Git Repository

```
git init
```

Step 5: Create .gitignore File

Create a file named .gitignore:

```
venv/
```

```
__pycache__/
```

```
.py
```

```
db.sqlite3
```

```
.env
```

Step 6: Check Git Status

```
git status
```

Step 7: Add Files to Git

`git add .`

Step 8: Commit the Project

`git commit -m "Initial commit of Doctor Finder Django project"`

Step 9: Create GitHub Repository

1. Open <https://github.com>
2. Click New Repository
3. Repository name:
doctor-finder
4. Select Public
5. Click Create Repository

Step 10: Connect Local Project to GitHub

`git remote add origin https://github.com/username/doctor-finder.git`

Check:

`git remote -v`

Step 11: Push Project to GitHub

`git branch -M main`

`git push -u origin main`

Step 12: Verify Deployment

Open GitHub repository

All Django files visible

Project successfully deployed

Future Updates

After changes:

git add .

git commit -m "Updated doctor profile module"

git push

18. Live Project Deployment (PythonAnywhere)

Q.18 Deploy a Django project to PythonAnywhere.

Ans.

Step 1: Create a PythonAnywhere Account

1. Go to <https://www.pythonanywhere.com>
2. Click Sign Up
3. Choose Beginner (Free) account
4. Login to dashboard

Step 2: Upload Project Code

Option A: Clone from GitHub

1. Open Bash Console in PythonAnywhere
2. Run:

git clone <https://github.com/username/doctor-finder.git>

Step 3: Create Virtual Environment

python3.10 -m venv venv

Activate it:

source venv/bin/activate

Step 4: Install Dependencies

pip install Django

Step 5: Configure Django Settings

Open settings.py:

```
ALLOWED_HOSTS = ['username.pythonanywhere.com']
```

Step 6: Configure Static Files

In settings.py:

```
STATIC_URL = '/static/'
```

```
STATIC_ROOT = BASE_DIR / 'staticfiles'
```

Run:

```
python manage.py collectstatic
```

Step 7: Database Migration

```
python manage.py migrate
```

Create admin user:

```
python manage.py createsuperuser
```

Step 8: Create a Web App

1. Go to Web tab
2. Click Add a new web app
3. Select:
 - Python version (3.10)
 - Manual Configuration
 - Django

Step 9: Configure WSGI File

Open WSGI configuration file:

Example:

```
import os
```

```
import sys
```

```
path = '/home/username/doctor-finder'
```

```
if path not in sys.path:
```

```
    sys.path.append(path)
```

```
os.environ['DJANGO_SETTINGS_MODULE'] = 'doctor_finder.settings'
```

```
from django.core.wsgi import get_wsgi_application
```

```
application = get_wsgi_application()
```

Step 10: Set Virtual Environment

In Web - Virtualenv:

```
/home/username/venv
```

Practical Example:

18) Write a Django project and deploy it on PythonAnywhere, making it accessible online.

Ans.

Step 1: Create Django Project

```
django-admin startproject doctor_finder
```

```
cd doctor_finder
```

```
python manage.py startapp doctors
```

Project Structure

```
doctor_finder/  
├── manage.py  
├── doctor_finder/  
│   ├── settings.py  
│   ├── urls.py  
│   └── wsgi.py  
└── doctors/  
    ├── views.py  
    └── templates/
```

Step 2: Create a Simple View

doctors/views.py

```
from django.http import HttpResponse  
  
def home(request):  
    return HttpResponse("Welcome to Doctor Finder")
```

Step 3: Configure URLs

doctor_finder/urls.py

```
from django.contrib import admin  
from django.urls import path  
from doctors.views import home  
  
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path("", home),  
]
```

Step 4: Run Project Locally

```
python manage.py runserver
```

Step 5: Push Project to GitHub

```
git init
```

```
git add .
```

```
git commit -m "Doctor Finder Django project"
```

```
git remote add origin https://github.com/username/doctor-finder.git
```

```
git push -u origin main
```

Step 6: Create PythonAnywhere Account

1. Visit <https://www.pythonanywhere.com>
2. Sign up (Free account)
3. Login to dashboard

Step 7: Clone Project on PythonAnywhere

Open Bash Console:

```
git clone https://github.com/username/doctor-finder.git
```

Step 8: Create Virtual Environment

```
python3.10 -m venv venv
```

```
source venv/bin/activate
```

```
pip install Django
```

Step 9: Configure Django Settings

settings.py

```
ALLOWED_HOSTS = ['username.pythonanywhere.com']
```

Static files:

```
STATIC_URL = '/static/'
```

```
STATIC_ROOT = BASE_DIR / 'staticfiles'
```

Run:

```
python manage.py collectstatic
```

```
python manage.py migrate
```

Step 10: Create Web App on PythonAnywhere

1 Go to Web tab

2 Click Add new web app

3 Choose:

- Python 3.10
- Manual configuration
- Django

Step 11: Configure WSGI File**WSGI Configuration**

```
import os
```

```
import sys
```

```
path = '/home/username/doctor-finder'
```

```
if path not in sys.path:
```

```
    sys.path.append(path)
```

```
os.environ['DJANGO_SETTINGS_MODULE'] = 'doctor_finder.settings'
```

```
from django.core.wsgi import get_wsgi_application
```

```
application = get_wsgi_application()
```

Step 12: Set Virtual Environment Path

/home/username/venv

Step 13: Reload Web App

Click Reload button.

Final Output (Online Access)

Website URL:

<https://username.pythonanywhere.com>

19. Social Authentication

Q.19 Implement Google and Facebook login for the Django project.

Ans.

Step 1: Install Required Packages

Activate virtual environment, then run:

pip install django-allauth

Step 2: Update INSTALLED_APPS

settings.py

```
INSTALLED_APPS = [  
    'django.contrib.sites',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',
```

```
'allauth',
'allauth.account',
'allauth.socialaccount',

'allauth.socialaccount.providers.google',
'allauth.socialaccount.providers.facebook',
]
```

Step 3: Configure Authentication Backends

settings.py

```
AUTHENTICATION_BACKENDS = (
    'django.contrib.auth.backends.ModelBackend',
    'allauth.account.auth_backends.AuthenticationBackend',
)
```

Step 4: Configure Sites Framework

settings.py

```
SITE_ID = 1
```

Run migrations:

```
python manage.py migrate
```

Step 5: Update Project URLs

urls.py

```
from django.contrib import admin
```

```
from django.urls import path, include
```

```
urlpatterns = [
```

```
path('admin/', admin.site.urls),
path('accounts/', include('allauth.urls')),
]
```

Step 6: Create Login Template

templates/account/login.html

```
<h2>Login</h2>
```

```
<a href="{% provider_login_url 'google' %}">Login with Google</a><br><br>
```

```
<a href="{% provider_login_url 'facebook' %}">Login with Facebook</a>
```

Step 7: Configure Google Login

1. Go to Google Developer Console
2. Create a new project
3. Enable Google+ / OAuth API
4. Create OAuth Client ID
5. Add redirect URI:

<http://localhost:8000/accounts/google/login/callback/>

Add Google App in Django Admin

1. Open /admin
2. Go to Social Applications
3. Add:
 - Provider: Google
 - Client ID
 - Secret Key
 - Sites: example.com

Step 8: Configure Facebook Login

1. Go to Facebook Developers
2. Create new app
3. Add Facebook Login

4. Redirect URI:

<http://localhost:8000/accounts/facebook/login/callback/>

Add Facebook App in Django Admin

- Provider: Facebook
- App ID
- App Secret
- Sites: example.com

Step 9: Configure Login Settings

settings.py

LOGIN_REDIRECT_URL = '/'

LOGOUT_REDIRECT_URL = '/'

Step 10: Run the Server

python manage.py runserver

Practical Example:

19) Write a Django project to allow users to log in using Google or Facebook.

Ans.

Step 1: Create Django Project

django-admin startproject social_login

cd social_login

python manage.py startapp accounts

Step 2: Install Required Package

pip install django-allauth

Step 3: Configure settings.py

Add Apps

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'django.contrib.sites',  
  
    'allauth',  
    'allauth.account',  
    'allauth.socialaccount',  
    'allauth.socialaccount.providers.google',  
    'allauth.socialaccount.providers.facebook',  
]
```

Authentication Backends

```
AUTHENTICATION_BACKENDS = (  
    'django.contrib.auth.backends.ModelBackend',  
    'allauth.account.auth_backends.AuthenticationBackend',  
)
```

Sites Framework

```
SITE_ID = 1
```

Redirect Settings

```
LOGIN_REDIRECT_URL = '/'
```

```
LOGOUT_REDIRECT_URL = '/'
```

Step 4: Run Migrations

```
python manage.py migrate
```

```
python manage.py createsuperuser
```

Step 5: Configure URLs

social_login/urls.py

```
from django.contrib import admin
```

```
from django.urls import path, include
```

```
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path('accounts/', include('allauth.urls')),  
]
```

Step 6: Create Login Template

templates/account/login.html

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <title>Social Login</title>
```

```
</head>
```

```
<body>
```

```
    <h2>Login Using</h2>
```

```
    <a href="{% provider_login_url 'google' %}">Login with  
Google</a><br><br>
```

```
    <a href="{% provider_login_url 'facebook' %}">Login with Facebook</a>
```

```
</body>
```

```
</html>
```

Step 7: Configure Google Login

1. Visit Google Developer Console
2. Create OAuth credentials
3. Add redirect URI:

<http://localhost:8000/accounts/google/login/callback/>

Step 8: Configure Facebook Login

1. Visit **Facebook Developers**
2. Create app and enable Facebook Login
3. Redirect URI:

<http://localhost:8000/accounts/facebook/login/callback/>

Step 9: Add Social Applications (Admin Panel)

1. Login to /admin
2. Go to **Social Applications**
3. Add:
 - Provider (Google / Facebook)
 - Client ID / App ID
 - Secret Key
 - Sites: example.com

Step 10: Run Server

```
python manage.py runserver
```

20. Google Maps API

Q.20 Use Google Maps API to display doctor locations in the "Doctor Finder" project.

Ans.

Step 1: Get Google Maps API Key

1. Go to <https://console.cloud.google.com>
2. Create a new project
3. Enable Maps JavaScript API

4. Generate API Key
5. Copy the API key

Step 2: Create Doctor Model with Location

doctors/models.py

```
from django.db import models
```

```
class Doctor(models.Model):  
    name = models.CharField(max_length=100)  
    specialty = models.CharField(max_length=100)  
    latitude = models.FloatField()  
    longitude = models.FloatField()  
  
    def __str__(self):  
        return self.name
```

Step 3: Apply Migrations

```
python manage.py makemigrations
```

```
python manage.py migrate
```

Step 4: Add Sample Doctor Data

Using Django shell:

```
python manage.py shell  
  
from doctors.models import Doctor
```

```
Doctor.objects.create(  
    name="Dr. Amit Sharma",  
    specialty="Cardiologist",
```

```
latitude=28.6139,  
longitude=77.2090  
)
```

```
Doctor.objects.create(  
    name="Dr. Neha Verma",  
    specialty="Dermatologist",  
    latitude=28.7041,  
    longitude=77.1025  
)
```

Step 5: Create View to Pass Doctor Data

doctors/views.py

```
from django.shortcuts import render  
from .models import Doctor  
def doctor_map(request):  
    doctors = Doctor.objects.all()  
    return render(request, 'doctors/map.html', {'doctors': doctors})
```

Step 6: Configure URLs

doctors/urls.py

```
from django.urls import path  
from .views import doctor_map  
  
urlpatterns = [  
    path('map/', doctor_map, name='doctor_map'),  
]
```

Project urls.py

```
path("", include('doctors.urls')),
```

Step 7: Create Google Map Template

templates/doctors/map.html

```
<!DOCTYPE html>

<html>

<head>

    <title>Doctor Locations</title>

    <style>

        #map {

            height: 500px;

            width: 100%;

        }

    </style>

</head>

<body>


    <h2>Doctor Finder – Locations</h2>

    <div id="map"></div>


    <script>

function initMap() {

    var centerLocation = { lat: 28.6139, lng: 77.2090 };


    var map = new google.maps.Map(document.getElementById('map'), {

        zoom: 10,
```

```

        center: centerLocation
    });

    {% for doctor in doctors %}
    new google.maps.Marker({
        position: { lat: {{ doctor.latitude }}, lng: {{ doctor.longitude }} },
        map: map,
        title: "{{ doctor.name }} ({{ doctor.specialty }})"
    });
    {% endfor %}
}
</script>

<script
src="https://maps.googleapis.com/maps/api/js?key=YOUR_API_KEY&callback=initMap"
    async defer></script>

</body>

</html>

```

Step 8: Run the Server

```
python manage.py runserver
```

Practical Example:

20) Write a Django project to display doctor locations using Google Maps API.

Ans.

Step 1: Create Django Project and App

```
django-admin startproject doctor_finder
```

```
cd doctor_finder
```

```
python manage.py startapp doctors
```

Step 2: Add App to Settings

settings.py

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'doctors',  
]
```

Step 3: Create Doctor Model

doctors/models.py

```
from django.db import models
```

```
class Doctor(models.Model):
```

```
    name = models.CharField(max_length=100)
```

```
    specialty = models.CharField(max_length=100)
```

```
latitude = models.FloatField()
longitude = models.FloatField()
```

```
def __str__(self):
    return self.name
```

Step 4: Apply Migrations

```
python manage.py makemigrations
python manage.py migrate
```

Step 5: Create View to Display Map

doctors/views.py

```
from django.shortcuts import render
from .models import Doctor
```

```
def doctor_map(request):
    doctors = Doctor.objects.all()
    return render(request, 'doctors/map.html', {'doctors': doctors})
```

Step 6: Configure URLs

doctors/urls.py

```
from django.urls import path
from .views import doctor_map
```

```
urlpatterns = [
    path('map/', doctor_map, name='doctor_map'),
]
```

Project urls.py

```
from django.contrib import admin
from django.urls import path, include
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path("", include('doctors.urls')),
]
```

Step 7: Create Google Maps Template

templates/doctors/map.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Doctor Finder - Locations</title>
    <style>
        #map {
            height: 500px;
            width: 100%;
        }
    </style>
</head>
<body>

<h2>Doctor Finder - Doctor Locations</h2>
<div id="map"></div>
```

```
<script>
function initMap() {
    var center = { lat: 28.6139, lng: 77.2090 };

    var map = new google.maps.Map(document.getElementById("map"), {
        zoom: 10,
        center: center
    });

    {% for doctor in doctors %}
    new google.maps.Marker({
        position: { lat: {{ doctor.latitude }}, lng: {{ doctor.longitude }} },
        map: map,
        title: "{{ doctor.name }} - {{ doctor.specialty }}"
    });
    {% endfor %}
}
</script>
```

```
<script
src="https://maps.googleapis.com/maps/api/js?key=YOUR_API_KEY&callback=initMap"
    async defer></script>
```

```
</body>
```

```
</html>
```

Step 8: Add Sample Doctor Data

```
python manage.py shell
```

```
from doctors.models import Doctor
```

```
Doctor.objects.create(name="Dr. Ravi Kumar", specialty="Cardiologist",  
latitude=28.6139, longitude=77.2090)
```

```
Doctor.objects.create(name="Dr. Anjali Singh", specialty="Dentist",  
latitude=28.7041, longitude=77.1025)
```

Step 9: Run Server

```
python manage.py runserver
```

Open in browser:

<http://127.0.0.1:8000/map/>